



WORK ATLAS PORTAL

Project Presentation Outline

Work Atlas Chicken, Waffles and Job Portal

AI-powered job matching platform connecting the right employer with the best candidate!

Team: -

Team 1 – Grace Li, Jack Markle, Michael Sanders

****** Gabriel Scruggs, Ahmed Kahl

Agenda & Timeline

- **Presentation: 20 Minutes:**
- **Q&A / Discussion: 10 Minutes:**
 1. **Opportunity** - Why we built this (~2-3 min)
 2. **Live Demo** - See it in action (~10 min)
 3. **The How** - Architecture & AI Engine (~5 min)
 4. **Business Model** - Cost simulation (~2 min)
 5. **Lessons Learned** - Reflections & next steps (~1 min)

The Opportunity (The “Why”)

- **Core Problem**
 - **Pain Points for Job Seekers:**
 - Manual resume submissions
 - Limited job match accuracy
 - Time-consuming application processes
 - **Pain Points for Employers:**
 - Difficulty finding qualified candidates
 - Manual resume screening
 - Inefficient hiring workflows
 - **Why a New Solution?**
 - Traditional job boards lack intelligent matching automation
 - Opportunity to improve experience for both sides

Live Demo

Live Application Walkthrough

- Job Seeker Journey:

1. Profile creation & resume upload
2. AI-powered resume parsing
3. Smart job recommendations
4. Application process

- Employer Journey:

1. Job posting creation
2. AI-enhanced job descriptions
3. Candidate matching
4. Application review & filtering

System Architecture - Frontend

The Technical Foundation (Part 1)

Frontend Architecture: - Component structure - State management - User flow diagrams - Technology stack: [Next.js, React, etc.]

[Insert: Frontend Architecture Diagram]

Key Features: - Responsive design - Real-time updates - Intuitive user interface

System Architecture - Backend

The Technical Foundation (Part 2)

Backend Architecture: - REST API design - Service layer architecture - Database communication - Authentication & authorization

[Insert: Backend Architecture Diagram]

Technology Stack: - FastAPI - MongoDB - n8n workflows - Authentication system

Data Model & Schema

Database Structure

MongoDB Schema: - User collections (Seekers & Employers) -
Job postings structure - Application tracking - AI metadata
storage

[Insert: ERD / MongoDB Schema Diagram]

How Systems Work Together: - Frontend → API → Services →
Database - Real-time data flow - Efficient query patterns

ChromaDB – 4 collections (2 for Job Seekers, 2 for Employers)

Slide 8: The AI Engine (The “How” - Part 2)

Smart Features Under the Hood

AI Processes Demonstrated:

- 1. Resume Parsing** - Extract structured data from unstructured resumes - Parse skills, experience, education - Technology: GPT-4o
 - 2. Job Matching & Recommendations** - Semantic matching algorithm - Skills alignment scoring - Experience relevance calculation
 - 3. Content Enhancement** - AI-generated job descriptions - Application response suggestions - Profile optimization tips
- Connection to Demo:** - “When I uploaded the resume, this parsing happened...” - “The recommendations you saw were generated by this matching logic...”

Slide 9: Business & AI Cost Simulation

Operational Costs & Scaling Model

Cost Breakdown:

Fixed Costs (Monthly): - Hosting: \$[X] - MongoDB Atlas: \$[X] - n8n Plan: \$[X] - **Total Fixed:** \$[X]

Variable AI Costs (Per Transaction): - Resume Parse (GPT-4o): ~2,500 tokens = **\$0.02** - AI Recommendation (GPT-4o): ~1,200 tokens = **\$0.008**

Scaling Simulation:

User Base Tier	Assumptions (per month)	Estimated Monthly AI Cost
Launch (100 Users)	20 new seekers200 recommendations	$(20 \times \$0.02) + (200 \times \$0.008) = \text{~}\mathbf{\$2.00/mo}$
Growth (1,000 Users)	200 new seekers2,000 recommendations	$(200 \times \$0.02) + (2,000 \times \$0.008) = \text{~}\mathbf{\$20.00/mo}$
Scale (10,000 Users)	2,000 new seekers20,000 recommendations	$(2,000 \times \$0.02) + (20,000 \times \$0.008) = \text{~}\mathbf{\$200.00/mo}$

Note: Estimates based on current OpenAI pricing and projected token usage

AI Cost-Control Strategy

Managing Scalability

Strategy for Sustainable Growth:

- 1. Process Once, Store Forever** - Cache parsed resume data - Store AI-generated content - Minimize redundant API calls
 - 2. Smart Caching** - Cache frequently requested recommendations - Implement intelligent refresh strategies - Redis/in-memory caching layer
 - 3. Tiered Model Usage** - GPT-4o for critical operations (parsing, matching) - GPT-3.5-turbo for lower-priority tasks - Consider open-source models for specific use cases
 - 4. Batch Processing** - Group similar operations - Optimize token usage - Reduce API overhead
- Cost Impact:** - Estimated 40-60% reduction in variable costs - Maintains performance and quality - Scalable architecture

Lessons Learned & Known Issues

Project Reflection

Key Technical Challenges Overcome: 1. [Challenge 1] - How you solved it - Example: Integrating real-time AI processing without blocking UI 2. [Challenge 2] - The solution - Example: Designing efficient vector embeddings for job matching

Known Limitations: - [Bug/Limitation 1] - [Bug/Limitation 2] - [Feature gap]

Next Steps: - Planned improvements - Future features - Optimization targets

Thank You & Discussion

Q&A

Thank you!

What questions do you have?

Additional Notes for Presenter:

Timing Checkpoints: - 3 min: Should be finishing “The Opportunity” - 13 min: Should be wrapping up Live Demo - 18 min: Should be finishing Architecture/AI - 20 min: Conclude presentation, open for Q&A

Demo Preparation: - Test all demo flows beforehand - Have backup screenshots ready - Prepare sample data that showcases best features

Anticipated Questions: - Security & data privacy - Scalability beyond 10k users - Competitive differentiation - Monetization strategy - Technology choices rationale